HackenProof

Expert crowdsourced security provider



Final Report

Private Security Audit

for **DarkSwap Curated SCA**

PREPARED BY

HackenProof

REPORT CREATED

29.10.2026

AUDIT DURATION

20.10.2026 - 26.10.2026

LANGUAGE

Solidity, Rust, Other

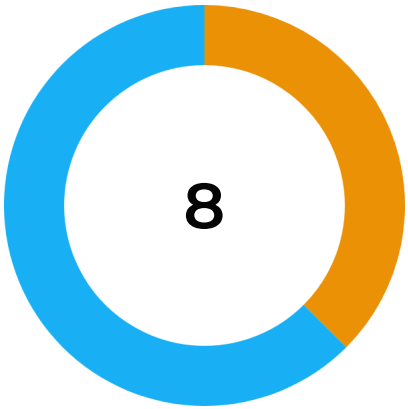
INTRODUCTION

HackenProof Audit is your solution to maximize cybersecurity through the involvement of the industry's most skilled and experienced community security researchers.

Program name	DarkSwap Curated SCA
Type	Private Security Audit
Duration	20.10.2026 - 26.10.2026
Changelog	26.10.2026 - Preliminary Report 29.10.2026 - Final Report
Scope	https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md
Approved by	HackenProof team
Audited by	 @shealtielanz  @tchkvsky

AUDIT SUMMARY

8	7	1
Total Findings	Resolved	Informative



Critical	0
High	0
Medium	3
Low	5
None	0

FINDINGS LIST

ID	Report name	Severity	Status
CUS-1	Untaxed relayer payments in `transact()` let note holders bypass the withdrawal f...	Medium	Resolved
CUS-2	Zero-value `transact()` proofs can exhaust Merkle capacity and halt deposits	Medium	Resolved
CUS-4	Malicious Users Can Bypass Protocol Fees Via Relayer Spoofing	Medium	Resolved
CUS-5	Gross deposit-cap checks reject proof deposits whose final note balance fits	Low	Resolved
CUS-6	Exact-delivery check in ` transferOut()` blocks withdrawals after a token fee switch	Low	Resolved
CUS-7	Unchecked handover in Solana `set_admin()` can lock out the pool admin	Low	Resolved
CUS-8	The guardian pause window is accepted unvalidated on both chains	Low	Resolved

ID	Report name	Severity	Status
CUS-9	Informational	Low	Informative

FINDING DETAILS

Untaxed relayer payments in `transact()` let note holders bypass the withdrawal fee

Report ID: CUS-1

Status: Resolved

Severity: Medium

TARGET

`https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md`

VULNERABILITY DETAILS

[M-1] Untaxed relayer payments in `transact()` let note holders bypass the withdrawal fee

Severity: Medium

Risk Category: Functional correctness

Context

- **File**: evm/src/DarkPool.sol
- **Function**: `transact()`, `_publicValues()`
- **Lines**: 398-411 (a nonzero `fee` with `extAmount == 0` is treated as value leaving the pool), 289-303 (the protocol fee is computed on `-extAmount` only; `fee` is paid untaxed), 278-285 (the deposit branch)
- **File**: solana/pool/src/lib.rs
- **Function**: `transact()`
- **Lines**: 753-765 (`public_amount` covers `|ext_amount| + fee`), 809-826 (`protocol_fee` is taken from `payout` only; `fee` goes to the relayer in full)
- **Related**: evm/src/DarkPool.sol:36-38 (the stated policy: a share of every shield and every unshield)

Description

The pool charges the withdrawal fee on `extAmount`, the amount sent to `recipient`. It does not charge that fee on `fee`, even though the relayer payment spends value from the same notes. A holder can set `extAmount` to zero, put the whole withdrawal in `fee`, and name an address they control as the relayer.

On the EVM, `_publicValues()` treats the transaction as a private transfer only when both values are zero; otherwise the notes give up `fee - extAmount` of the public asset:

```
function _publicValues(ExtData calldata ext) internal view returns (uint256 public
    if (ext.extAmount == 0 && ext.fee == 0) {
        if (ext.token != address(0)) revert BadExtData();
        return (0, 0);
    }
    // ...
    int256 net = ext.extAmount - int256(ext.fee);
    if (net == 0) revert BadExtData();
    publicAmount = net >= 0 ? uint256(net) : FIELD - uint256(-net);
    // ...
}
```

The value section charges `feeOn` only inside the `extAmount < 0` branch and pays `fee` afterwards with no fee entry:

```
function transact(Proof calldata proof, ExtData calldata ext) external payable nor
    // ...
    if (ext.extAmount < 0) {
        uint256 payout = uint256(-ext.extAmount);
        uint256 fee = feeOn(payout);
        _chargeFee(1, ext.token, fee);
        _pay(asset, ext.token, ext.recipient, payout - fee);
    }
    if (ext.fee > 0) _pay(asset, ext.token, ext.relayer, ext.fee);
}
```

Solana follows the same accounting: `moves_value` is true when `ext_amount != 0 || fee != 0`, and `public_amount` includes both the payout and relayer fee. Yet `protocol_fee(pool, payout)` uses only `payout = ext_amount.unsigned_abs()`. The EVM test reproduces the bypass; the Solana path was checked in source code.

The proof binds the net public amount, so `extAmount = 0, fee = amount` spends the same note value as `extAmount = -amount, fee = 0`. The wallet rejects a zero payout, but the contracts do not. At tree capacity, both pools reject `extAmount == 0`; a one-unit payout with the rest routed through `fee` still avoids nearly all of the withdrawal fee.

The EVM also allows a deposit alongside a larger relayer payment. In `test_M1c`, a 0.001 ETH deposit and a 1.001 ETH self-relayer payment spend a 1 ETH note while accruing only the deposit fee. Solana rejects a positive `ext_amount`, so this variant is EVM-only.

Impact

Any note holder can avoid the withdrawal fee by directing the full payout to a self-controlled relayer. At the planned 50 bps rate, the fee recipient loses up to 0.005 ETH per 1 ETH withdrawn this way; deposit fees still accrue. The bypass persists in the immutable EVM deployment and the corresponding Solana fee path. User principal and pool solvency are unaffected.

Proof of Concept

Scenario (`test_M1a`):

1. The pool is deployed with the repository's `Groth16Verifier` at 50 bps, ETH is listed, and Alice shields 1 ETH paying 1.005 ETH to a note whose key the withdrawer holds. `protocolFees(ETH)` is 0.005 ETH.
2. The withdrawer builds a real proof with the dev keys for `ext = {recipient: 0, extAmount: 0, relayer: <their own address>, fee: 1 ETH, token: ETH}` and submits `transact` from that address.
3. That address receives 1 ETH, the ETH note books go to zero, and `protocolFees(ETH)` is still 0.005 ETH.
4. In `test_M1b` the same note is withdrawn with `extAmount = -1 ETH, fee = 0` to the same address: it receives 0.995 ETH and the fee books gain 0.005 ETH.
5. In `test_M1c` the holder submits a real proof with `extAmount = 0.001 ETH` and `fee = 1.001 ETH` to their own relayer address: the relayer receives 1.001 ETH, the 1 ETH note is consumed, and 0.000005 ETH of protocol fee accrues.

Tests in the attached `test/DarkPoolFindingsPOC.t.sol`:

- `test_M1a_RelayerFeeOnlyWithdrawalPaysNoProtocolFee`
- `test_M1b_DocumentedWithdrawalOfTheSameNoteIsCharged`
- `test_M1c_MinDepositThroughProofRoutesFullNoteValueToRelayer`

The commands to run these tests and the observed output are in validation-steps.md, section M-1.

Recommendation

The fee should be computed on the net value leaving the notes' books, not on the recipient's share alone. That base is `payout + fee` for a withdrawal and `max(fee - extAmount, 0)` for an EVM mixed deposit and relayer payment. A helper can make the fee base explicit:

```
function _noteOutflow(ExtData calldata ext) internal pure returns (uint256) {
    if (ext.extAmount < 0) return uint256(-ext.extAmount) + ext.fee;
    uint256 deposit = uint256(ext.extAmount);
    return ext.fee > deposit ? ext.fee - deposit : 0;
}
```

Charge `feeOn(_noteOutflow(ext))` and split the charge between recipient and relayer payments while preserving the proof-bound total note debit and asset ledger. A relayer can reject a quote when its payment would be reduced. Apply the same split in the Solana payout path and add regression tests for zero recipient payout and the EVM mixed-deposit form.

VALIDATION STEPS

M-1. Untaxed relayer payments in `transact()` let note holders bypass the withdrawal fee

1. Check evm/src/DarkPool.sol:398-411: `_publicValues` returns a nonzero `publicAmount` whenever `extAmount` or `fee` is nonzero; the notes' balance changes by `extAmount - fee`, an outflow when negative.

2. Check evm/src/DarkPool.sol:289-303: `feeOn` is applied to `-extAmount` only inside the `extAmount < 0` branch; `ext.fee` is paid with `_pay` and no `_chargeFee`.

3. Check evm/src/DarkPool.sol:278-285: the deposit branch charges `feeOn(extAmount)` on the gross deposit and then pays `ext.fee` from the asset balance, so a `fee` larger than `extAmount` drains a note untaxed.

4. Check solana/pool/src/lib.rs:753-765 and 809-826: `public_amount` covers `|ext_amount| + fee`; `protocol_fee` is computed from `payout` alone; `fee` is paid in full.

5. Read the proof. The 3 functions below are taken verbatim from `test/DarkPoolFindingsPOC.t.sol`:

```
function test_M1a_RelayerFeeOnlyWithdrawalPaysNoProtocolFee() public {
    setUpM1();
    uint256 feesBefore = m1Pool.protocolFees(ETH);
    assertEq(feesBefore, 0.005 ether, "only the shield fee has accrued");

    (DarkPool.Proof memory p, DarkPool.ExtData memory e) = FeeBypassProof.feeO
    assertEq(e.extAmount, 0, "nothing is declared as leaving to a recipient");
    assertEq(e.fee, 1 ether, "the whole note is declared as the relayer fee");

    vm.prank(FeeBypassProof.ATTACKER);
    m1Pool.transact(p, e);

    assertEq(FeeBypassProof.ATTACKER.balance, 1 ether, "the full 1 ETH note le
```

```
    assertEq(m1Pool.protocolFees(ETH), feesBefore, "no unshield fee was charge
    assertEq(noteBooks(m1Pool, ETH), 0, "the note books were debited in full")
}
```

```
function test_M1b_DocumentedWithdrawalOfTheSameNoteIsCharged() public {
    setUpM1();
    uint256 feesBefore = m1Pool.protocolFees(ETH);

    (DarkPool.Proof memory p, DarkPool.ExtData memory e) = FeeBypassProof.docu
    assertEq(e.extAmount, -1 ether, "the same 1 ETH declared as a payout");

    vm.prank(FeeBypassProof.ATTACKER);
    m1Pool.transact(p, e);

    assertEq(FeeBypassProof.ATTACKER.balance, 0.995 ether, "payout minus the 0
    assertEq(m1Pool.protocolFees(ETH) - feesBefore, 0.005 ether, "the unshield
}
```

```
function test_M1c_MinDepositThroughProofRoutesFullNoteValueToRelayer() public
    setUpM1();
    (DarkPool.Proof memory p, DarkPool.ExtData memory e) = M1DepositThroughPro
    uint256 feesBefore = m1Pool.protocolFees(ETH);
    uint256 relayerBefore = M1DepositThroughProof.RELAYER.balance;
    uint256 deposit = uint256(e.extAmount);

    vm.prank(alice);
    m1Pool.transact{value: deposit + m1Pool.feeOn(deposit)}(p, e);

    assertEq(M1DepositThroughProof.RELAYER.balance - relayerBefore, e.fee, "th
    assertEq(noteBooks(m1Pool, ETH), 0, "the 1 ETH note is fully consumed");
    assertEq(m1Pool.protocolFees(ETH) - feesBefore, m1Pool.feeOn(deposit), "on
    assertEq(address(m1Pool).balance, m1Pool.protocolFees(ETH), "only shield a
    assertTrue(m1Pool.nullifierSpent(p.inputNullifiers[0]));
    assertTrue(m1Pool.nullifierSpent(p.inputNullifiers[1]));
}
```

6. Run the proof. Expected: 3 passed, 0 failed.

```
forge test --match-path test/DarkPoolFindingsPOC.t.sol --match-test "test_M1a_
```

Observed output on the commit above:

```
[PASS] test_M1a_RelayerFeeOnlyWithdrawalPaysNoProtocolFee()
[PASS] test_M1b_DocumentedWithdrawalOfTheSameNoteIsCharged()
[PASS] test_M1c_MinDepositThroughProofRoutesFullNoteValueToRelayer()
Suite result: ok. 3 passed; 0 failed; 0 skipped
```

7. The team's closest test, at evm/test/DarkPool.t.sol:380-402, always pairs the relayer fee with a nonzero payout, so it never reaches the fee-only shape; the invariant handler in `SolvencyInvariant.t.sol` bounds its withdrawal amount to at least 1, for the same reason.

FINDING DETAILS

Zero-value `transact()` proofs can exhaust Merkle capacity and halt deposits

Report ID: CUS-2

Status: Resolved

Severity: Medium

TARGET

`https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md`

VULNERABILITY DETAILS

[M-2] Zero-value `transact()` proofs can exhaust Merkle capacity and halt deposits

Severity: Medium

Risk Category: Griefing

Context

- **File**: evm/src/MerkleTree.sol
- **Function**: `_insertPair()`
- **Lines**: 14 (`MAX_LEAVES` , 262,144 leaves), 57-60 (two leaves per insert; `TreeFull` once `index + 2 > MAX_LEAVES`)
- **File**: evm/src/DarkPool.sol
- **Function**: `shield()` , `transact()`
- **Lines**: 211 (a deposit inserts `(commitment, ZERO_LEAF)`), 232-243 (the capacity exit policy), 83 (the comment that `minDeposit` makes filling the tree costly)
- **Related**: circuits/transaction.circom:103-108 (membership is enforced only for inputs with a nonzero amount), solana/verifier-parity/src/merkle.rs:19-21 (same `MAX_LEAVES` ; a 256-root ring), solana/pool/src/lib.rs:691 (same zero-leaf padding on `shield`)

Description

Both pools cap the Merkle tree at 262,144 leaves and insert two leaves at a time. A `transact` inserts two output commitments. A `shield` inserts one deposit commitment and pads the second slot with `ZERO_LEAF`:

```
function shield(address token, uint256 amount, uint256 publicKey, uint256 blinding
    external payable nonReentrant returns (uint256 index)
{
    // ...
    uint256 commitment = commitmentOf(amount, assetIdOf(token), publicKey, blindir
    index = _insertPair(commitment, ZERO_LEAF);
    // ...
}
```

That leaves 131,072 pair insertions before `_insertPair` reverts with `TreeFull`. Once the tree is full, the pool accepts only whole-note public exits. It cannot accept another deposit or private transfer, and the EVM deployment has no reset or upgrade path.

The comment at `DarkPool.sol:83` says the minimum deposit makes a tree fill costly. A private transfer bypasses that minimum. The circuit checks Merkle membership only when an input amount is nonzero, so two zero-value inputs can produce two zero-value outputs against any known root. `test_M2c` submits a real proof of that transfer to an empty pool with no listed assets and deposits paused. It consumes two leaves without moving value or paying a protocol fee.

Even a deposit-based fill does not put the attacker's principal at risk: whole-note exits still work after the tree is full. At 50 bps and the 0.001 ETH minimum, the round-trip protocol fees for 131,072 deposits would total 1.31072 ETH, plus gas. `test_M2b` checks the last insertion and the exit path at the capacity boundary. The local zero-value transaction used about 2.61 million gas. Repeating it 131,072 times would use about 343 billion gas, or 3.4 ETH at a hypothetical 0.01 gwei. This is an estimate, not a full attack trace or a live gas quote. The attacker would also need to generate proofs and get the transactions included.

The EVM keeps the latest 1,000 roots. A local regression confirms that 1,000 later insertions make a proof against an older root fail, while a proof against the current root succeeds. Repeated disruption on a live chain would depend on block capacity and proving speed. On Solana, each `transact` creates two zero-data nullifier accounts. At 890,880 lamports of rent per account, a transfer-based fill would tie up about 233 SOL in rent; a deposit-and-exit fill would tie up about 117 SOL. The program retains 256 roots. These are code-derived estimates; no Solana runtime campaign was run.

Impact

Someone able to generate and submit enough proofs can end deposits and private transfers on a pool without depositing value or paying a protocol fee. Existing holders can still make whole-note

public exits, which expose timing and amounts. Continued private use requires a new deployment. The full 131,072-transaction campaign was not run.

Proof of Concept

The tests cover the zero-value transaction and the capacity boundary separately:

1. `test_M2c` submits a real Groth16 proof to an empty pool with deposits paused and no assets listed. `nextIndex` advances from 0 to 2; no value enters and no fee accrues.
2. `test_M2a` confirms that a shield also uses two leaves because it pads the deposit with `ZERO_LEAF`.
3. `test_M2b` places `nextIndex` at `MAX_LEAVES - 2` to simulate the last free pair, then makes a minimum deposit. Further deposits revert with `TreeFull`, while a whole-note public exit still succeeds. This boundary test uses a stub verifier and does not execute the full fill.

Tests in the attached `test/DarkPoolFindingsPOC.t.sol`:

- `test_M2a_EveryShieldConsumesTwoLeavesAndTheTreeHoldsOnlyHalfAsManyDeposits`
- `test_M2b_FillingTheTreeStopsDepositsForeverWhileTheFillerExitsAtTheFullPrincipal`
- `test_M2c_ZeroNoteTransferFillsAPairWithNoDepositAndNoFeeEvenWhileDepositsArePaused`

The commands to run these tests and the observed output are in validation-steps.md, section M-2.

Recommendation

First, constrain the circuit so a private transfer cannot have only zero-value inputs and outputs; require a spent nonzero note or a public deposit, then regenerate the proving and verifying artifacts. Separately, stop wasting a leaf on every deposit by buffering one pending deposit commitment and inserting two deposits per pair, or by supporting single-leaf inserts for deposits. Either change must preserve the root format proved by the circuit.

Plan for capacity exhaustion as well. The team could raise `MAX_LEAVES` within the depth-26 tree's range, with matching client recovery limits, or provide a migration path to a successor pool so holders need not exit publicly. If the limit stays, remove the claim that `minDeposit` prices a fill and document the actual gas and Solana rent costs. A fixed fee on private transfers would make zero-value fills more expensive; a larger Solana root history would give users more time to settle proofs.

VALIDATION STEPS

M-2. Zero-value `transact()` proofs can exhaust Merkle capacity and halt deposits

1. Check evm/src/MerkleTree.sol:14 (`MAX_LEAVES` , 262,144 leaves) and 57-60 (two leaves per insert, `TreeFull` at `index + 2 > MAX_LEAVES`).

2. Check evm/src/DarkPool.sol:211: `shield` inserts `(commitment, ZERO_LEAF)` ; and line 83: the comment that `minDeposit` makes filling the tree costly.

3. Check evm/src/DarkPool.sol:232-243: at capacity `transact` accepts only `extAmount < 0` with both outputs equal to the canonical zero commitment, so a filler's whole-note exits succeed; and 273-276: a private transfer returns before any `_chargeFee` , after its outputs were inserted at line 267.

4. Check circuits/transaction.circom:103-108: membership is enforced through `ForceEqualIfEnabled` with `enabled = inAmount` , so a zero-amount input needs no path and no note.

5. Check solana/verifier-parity/src/merkle.rs:19-21 and 127-129 (same bound; a 256-root ring overwritten on every insert), solana/pool/src/lib.rs:691 (same zero-leaf padding) and 788-794 (two zero-data nullifier accounts per `transact` , never closed).

6. Read the proof. The 3 functions below are taken verbatim from `test/DarkPoolFindingsPOC.t.sol` :

```
function test_M2c_ZeroNoteTransferFillsAPairWithNoDepositAndNoFeeEvenWhileDepo
    // A fresh pool with the real verifier, nothing listed, nothing deposited.
    Groth16Verifier verifier = new Groth16Verifier();
    vm.prank(FeeBypassProof.DEPLOYER);
    DarkPool pool = new DarkPool(address(verifier), address(0), block.timestam
    assertEq(address(pool), FeeBypassProof.POOL, "pool address bound by the pr
    assertEq(pool.root(), GhostTransferProof.EMPTY_ROOT, "the proof was built
    assertEq(pool.nextIndex(), 0);
    // Deposits are paused: no value can enter, and a shield would revert.
    vm.prank(owner);
    pool.setDepositsPaused(true);
    assertTrue(pool.depositsPaused());

    // Two zero-amount inputs need no Merkle membership, so the proof needs no
    (DarkPool.Proof memory p, DarkPool.ExtData memory e) = GhostTransferProof.
    address filler = address(0xF111E7);
    vm.prank(filler);
    uint256 gasBefore = gasleft();
    pool.transact(p, e);
    uint256 transactGas = gasBefore - gasleft();
    emit log_named_uint("gas per zero-note transfer", transactGas);

    assertEq(pool.nextIndex(), 2, "a pair of leaves is consumed");
    assertEq(pool.protocolFees(ETH), 0, "no fee of any kind");
    assertTrue(pool.depositsPaused(), "the pause never applied to this path");
    assertEq(address(pool).balance, 0, "no value entered the pool");
```

```
    assertTrue(pool.nullifierSpent(p.inputNullifiers[0]), "the filler's own fr
    assertLt(transactGas, 3_500_000, "one pair for a few million gas");
}
```

```
function test_M2b_FillingTheTreeStopsDepositsForeverWhileTheFillerExitsAtTheFu
    setUpM2();
    // Jump to the last free pair instead of executing 131,071 deposits.
    // slot 0 of MerkleTree is nextIndex.
    vm.store(address(m2Pool), bytes32(uint256(0)), bytes32(m2Pool.MAX_LEAVES()
    assertEq(m2Pool.nextIndex(), m2Pool.MAX_LEAVES() - 2);

    shieldMin(m2Pool, 3); // the filler's last deposit
    assertEq(m2Pool.nextIndex(), m2Pool.MAX_LEAVES(), "tree is full");

    // Nobody can deposit again: shield and deposit-through-proof both stop.
    uint256 charged = MIN_DEPOSIT + m2Pool.feeOn(MIN_DEPOSIT);
    vm.prank(alice);
    vm.expectRevert(MerkleTree.TreeFull.selector);
    m2Pool.shield{value: charged}(ETH, MIN_DEPOSIT, 7, 4, "");

    DarkPool.Proof memory p;
    p.root = m2Pool.root();
    p.inputNullifiers = [uint256(1), uint256(2)];
    p.outputCommitments = [uint256(111), uint256(222)];
    DarkPool.ExtData memory dep = DarkPool.ExtData({
        recipient: address(0), extAmount: int256(uint256(MIN_DEPOSIT)), relaye
        token: ETH, encryptedOutput1: "", encryptedOutput2: ""
    });
    vm.prank(alice);
    vm.expectRevert(MerkleTree.TreeFull.selector);
    m2Pool.transact{value: charged}(p, dep);

    // The owner has no remedy: no upgrade path, no tree reset.
    // The filler takes the principal back through a whole-note exit.
    uint256 zero = m2Pool.commitmentOf(0, m2Pool.assetIdOf(ETH), 0, 0);
    p.outputCommitments = [zero, zero];
    address filler = address(0xF111);
    DarkPool.ExtData memory exit = DarkPool.ExtData({
        recipient: filler, extAmount: -int256(uint256(MIN_DEPOSIT)), relayer:
        token: ETH, encryptedOutput1: "", encryptedOutput2: ""
    });
    m2Pool.transact(p, exit);
    assertEq(filler.balance, MIN_DEPOSIT - m2Pool.feeOn(MIN_DEPOSIT), "princip

    // Irrecoverable cost of filling the whole tree at the minimum deposit:
    // the deposit fee and the withdrawal fee on 131,072 deposits.
    uint256 roundTripFee = m2Pool.feeOn(MIN_DEPOSIT) * 2;
    uint256 totalFee = roundTripFee * (m2Pool.MAX_LEAVES() / 2);
    assertEq(totalFee, 1.31072 ether, "about 1.31 ETH in fees to retire the po
}
```

```
function test_M2a_EveryShieldConsumesTwoLeavesAndTheTreeHoldsOnlyHalfAsManyDep
    setUpM2();
```

```
        assertEq(m2Pool.nextIndex(), 0);
        shieldMin(m2Pool, 1);
        assertEq(m2Pool.nextIndex(), 2, "one deposit advanced the tree by two leav
        uint256 gasBefore = gasleft();
        shieldMin(m2Pool, 2);
        uint256 shieldGas = gasBefore - gasleft();
        emit log_named_uint("gas per minimum deposit", shieldGas);
        assertLt(shieldGas, 3_000_000, "a minimum deposit costs under three millio
        assertEq(m2Pool.MAX_LEAVES() / 2, 131_072, "deposits that fit before the t
    }
```

7. Run the proof. Expected: 3 passed, 0 failed. `test_M2c` uses the real verifier; `test_M2a` and `test_M2b` use a stub that accepts every proof.

```
forge test --match-path test/DarkPoolFindingsPOC.t.sol --match-test "test_M2a_
```

   Observed output on the commit above:

```
[PASS] test_M2a_EveryShieldConsumesTwoLeavesAndTheTreeHoldsOnlyHalfAsManyDepos
  gas per minimum deposit: 2508375
[PASS] test_M2b_FillingTheTreeStopsDepositsForeverWhileTheFillerExitsAtTheFull
[PASS] test_M2c_ZeroNoteTransferFillsAPairWithNoDepositAndNoFeeEvenWhileDeposi
  gas per zero-note transfer: 2613968
Suite result: ok. 3 passed; 0 failed; 0 skipped
```

8. The team's capacity tests (evm/test/DarkPool.t.sol:253-317 and the e2e scripts) set the fullness counter and exercise the exit policy; none of them inserts a pair without a deposit, so the free fill is not covered.

FINDING DETAILS

Malicious Users Can Bypass Protocol Fees Via Relayer Spoofing

Report ID: CUS-4

Status: Resolved

Severity: Medium

TARGET

`https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md`

VULNERABILITY DETAILS

Severity: Medium.
Affected files: `packages/darkswap-pool/evm/src/DarkPool.sol:289-302,398-410`
Parallel source path: `packages/darkswap-pool/solana/pool/src/lib.rs:753-765,809-837`

Summary

A note owner can bypass the protocol fee by labeling their own withdrawal as relayer compensation and receive the full amount without paying the withdrawal protocol fee. This bypasses the stated unshielded fee policy.

Vulnerability Details

Root cause: The fee base includes only the recipient withdrawal amount and excludes the owner-selected relayer payment. The core does not enforce that this excluded payment represents genuine relay compensation.

EVM: `transact` charges withdrawal fees only when `ext.extAmount < 0`. It pays `ext.fee` separately without that charge or an authenticated service-cost limit.

`DarkPool.sol:289-303`:

```
if (ext.extAmount < 0) {
    if (ext.recipient == address(0)) revert BadExtData();
    uint256 payout = uint256(-ext.extAmount);
    uint256 fee = feeOn(payout);
    // The fee comes out of the payout: the notes gave up `payout`,
    // the recipient receives `payout - fee`.
    if (payout > asset.balance) revert InsufficientPoolBalance();
    asset.balance -= uint128(fee);
    _chargeFee(1, ext.token, fee);
    _pay(asset, ext.token, ext.recipient, payout - fee);
}
if (ext.fee > 0) {
    if (ext.relayer == address(0)) revert BadExtData();
    _pay(asset, ext.token, ext.relayer, ext.fee);
}
```

The owner can generate a valid proof with:

```
extAmount = 0
fee       = amount being redeemed
relayer   = an owner-controlled address
```

The proof consumes the note value through `extAmount - fee`. The contract skips the withdrawal-fee branch and pays the owner-controlled relayer address.

`DarkPool.sol:406–410` binds that debit and the asset:

```
int256 net = ext.extAmount - int256(ext.fee);
// With net == 0 the circuit would not tie the notes to this token.
if (net == 0) revert BadExtData();
publicAmount = net >= 0 ? uint256(net) : FIELD - uint256(-net);
publicAssetId = assetIdOf(ext.token);
```

At 50 bps:

Redeeming 1 ETH	Owner receives	New protocol fee
Ordinary withdrawal	0.995 ETH	0.005 ETH
Owner acting as relayer	1 ETH	0

Legitimate relay compensation is intentionally exempt, but an entire owner-directed redemption can use the same exemption.

Solana counterpart: The public note debit includes both amounts (`pool/src/lib.rs:762`):

```
let out = (ext_amount.unsigned_abs() as u128) + fee as u128; // < 2^65
```

The protocol fee uses only the recipient payout (`lib.rs:810-814`):

```
let payout = ext_amount.unsigned_abs();
// The protocol fee comes out of the payout: the notes gave up
// `payout`, the recipient receives `payout - protocol_fee`.
let protocol = protocol_fee(pool, payout)?;
let amount = payout - protocol;
```

The relay payment is separate for SOL (`lib.rs:825-826`):

```
pay_lamports(vault, recipient, amount)?;
pay_lamports(vault, relayer, fee)?;
```

And for classic SPL tokens (`lib.rs:836-838`):

```
if fee > 0 {
    token_transfer(token_program, vault, mint_info, relayer, pool, fee, decimals,
}
```

Impact

The protocol loses withdrawal fee revenue.

VALIDATION STEPS

Check POC.

FINDING DETAILS

Gross deposit-cap checks reject proof deposits whose final note balance fits

Report ID: CUS-5

Status: Resolved

Severity: Low

TARGET

`https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md`

VULNERABILITY DETAILS

Revision: `03620857523a710db49c2164e7e0d9cebc051f56`
Severity: Low
Affected file: `packages/darkswap-pool/evm/src/DarkPool.sol`, lines 278–285, 300–302, 423–428 and 447–450.

Summary

`transact` checks the asset's deposit cap against the gross deposit before deducting the authorized relayer payment. A proof deposit can therefore revert even when the transaction's final note liability would fit the cap.

Vulnerability Details

Root cause: `_credit` checks and stores `existing balance + gross deposit` before `_pay` deducts the relayer payment. The cap check consequently ignores that later debit. The proof's public amount already includes the relay deduction, but deposit admission uses the gross amount.

In `DarkPool.sol:278-285`, the positive deposit branch credits the full external amount:

```
if (ext.extAmount > 0) {
    if (!asset.depositsEnabled || depositsPaused()) revert DepositsAreOff();
    _screen(msg.sender);
    uint256 amount = uint256(ext.extAmount);
    uint256 fee = feeOn(amount);
    _pull(ext.token, amount + fee, true);
    _credit(asset, amount);
    _chargeFee(2, ext.token, fee);
}
```

`DarkPool.sol:423–428` applies the cap immediately:

```
function _credit(Asset storage asset, uint256 amount) internal {
    if (amount < asset.minDeposit) revert UnderDepositMinimum();
    if (amount > asset.maxDeposit) revert OverDepositLimit();
    uint256 next = uint256(asset.balance) + amount;
    if (next > asset.depositCap) revert OverDepositCap();
    asset.balance = uint128(next);
}
```

Only afterward does `DarkPool.sol:300–302` pay the relayer:

```
if (ext.fee > 0) {
    if (ext.relayer == address(0)) revert BadExtData();
    _pay(asset, ext.token, ext.relayer, ext.fee);
}
```

That payment reduces note backing through `DarkPool.sol:447–450`:

```
function _pay(Asset storage asset, address token, address to, uint256 amount) inte
    if (amount > asset.balance) revert InsufficientPoolBalance();
    asset.balance -= uint128(amount);
    _transferOut(token, to, amount);
}
```

For starting note liability `B`, gross deposit `A`, relayer payment `R` and cap `K`, successful settlement ends at `B + A - R`. The current admission check instead requires `B + A <= K`.

Recorded example:

Quantity	Amount
Starting note liability	1 ETH
Gross deposit	0.4 ETH

Quantity	Amount
Authorized relayer payment	0.1 ETH
Final note liability if settled	1.3 ETH
Configured deposit cap	1.3 ETH
Intermediate amount checked	1.4 ETH
Result	`OverDepositCap`

The shield protocol fee is charged separately on the gross deposit and is excluded from note liability. At 50 bps, the additional deposit fee is 0.002 ETH; it does not account for the 0.1 ETH difference above.

Impact

Depositors can be unable to execute the combined deposit-and-relayer operation when `B + A > K` but `B + A - R <= K`, despite a valid proof and sufficient backing.

Recommendation

If the cap should limit final note liabilities, adjust only positive `transact` deposits:

1. Keep minimum/maximum checks on the gross deposit.
2. Calculate `B + A - R` in `uint256`, rejecting `R > B + A` or a result above the cap before casting to `uint128`.
3. Store that final balance once. Pay the relayer through `_transferOut` in this path, avoiding a second balance deduction through `_pay`.

Preserve existing deposit/relayer validation, exact funding, shielding, protocol fees, and other withdrawal paths. If gross admission is intentional, document that policy instead. This recommendation has not been implemented or retested.

VALIDATION STEPS

Not Needed.

FINDING DETAILS

Exact-delivery check in `_transferOut()` blocks withdrawals after a token fee switch

Report ID: CUS-6

Status: Resolved

Severity: Low

TARGET

`https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md`

VULNERABILITY DETAILS

[L-1] Exact-delivery check in `_transferOut()` blocks withdrawals after a token fee switch

Severity: Low

Risk Category: Conditional freezing of funds

Context

- **File**: evm/src/DarkPool.sol
- **Function**: `_transferOut()`
- **Lines**: 462-470 (both the pool's and the recipient's balance change must equal `amount`), 453-455 (the stated reason for the check)
- **Related**: evm/src/DarkPool.sol:81 (once listed, withdrawals always work), evm/src/DarkPool.sol:44-49 (no rescue, no upgrade path), evm/test/DarkPool.t.sol:695-704 (the team's test that expects the revert), docs/zk-pool-audit/threat-model.md:29 (exact-transfer policy)

Description

Every token payout, relayer fee and fee sweep goes through `_transferOut`, which requires two exact equalities after the transfer:

```
function _transferOut(address token, address to, uint256 amount) internal {
    // ...
    uint256 beforePool = IERC20(token).balanceOf(address(this));
    uint256 beforeRecipient = IERC20(token).balanceOf(to);
    IERC20(token).safeTransfer(to, amount);
    uint256 afterPool = IERC20(token).balanceOf(address(this));
    uint256 afterRecipient = IERC20(token).balanceOf(to);
    if (
        afterPool > beforePool || beforePool - afterPool != amount || afterRecipie
            || afterRecipient - beforeRecipient != amount
    ) revert BadTransferAmount();
}
```

The pool-side balance check protects other note holders: a token fee charged to the sender would otherwise reduce their backing. The recipient check enforces exact delivery. If a listed token later deducts a fee from what the recipient receives while debiting the pool by the full amount, the pool-side check passes and the recipient check fails. Every withdrawal and fee sweep of that token then reverts.

The pool has no way to relax the check or rescue the asset. `updateAsset` can change deposit limits, but it cannot change transfer behavior. Access returns if the issuer turns the token fee off. The team pins this behavior on purpose: `test_outgoingRecipientTaxCannotSilentlyShortPayWithdrawal` builds the same token shape and expects the revert, the threat model places fee-on-transfer tokens under an exact-transfer policy that rejects unsupported behavior, and the team's earlier review filed issuer-controlled freezes of a listed token as a listing-policy note. The behavior is therefore known and intended. What this finding adds is that the recipient-side half of the check protects no other holder, so the policy turns a token fee the recipient could have accepted into a loss of access for every holder of the asset while the fee stays on.

Impact

If the issuer enables a recipient-side fee after listing, holders cannot withdraw that asset and the pool cannot sweep its accrued fees. The lock lasts until the issuer disables the fee or the pool gains a new recovery path. Other assets are unaffected, and the pool remains solvent in its own books.

Proof of Concept

Scenario (`test_L1a`):

1. A pool at 50 bps lists an exact-transfer ERC-20. Alice shields 100 tokens; the pool holds 100.5 tokens.
2. The issuer enables a recipient-side fee of 1%.

3. A withdrawal of 10 tokens to a fresh address reverts with `BadTransferAmount`. After the 0.5% protocol fee the pool transfers 9.95 tokens, and the 1% recipient-side fee leaves the recipient with 9.8505, so the recipient-side equality fails.
4. `collectFees` for the token reverts for the same reason.
5. The owner disables deposits for the token; the withdrawal still reverts. The pool still holds 100.5 tokens and the note books still say 100.

Tests in the attached `test/DarkPoolFindingsPOC.t.sol`:

- `test_L1a_RecipientSideTokenFeeLocksEveryNoteOfThatAssetWhileThePoolStaysSolvent`

The commands to run this test and the observed output are in validation-steps.md, section L-1.

Recommendation

Keep exact receipt as the default, as the team's test requires. If the pool is meant to offer an exit when a listed token later adds a recipient-side fee, let the holder opt in to a bounded shortfall through a flag and limit bound into `extDataHash`. The pool-side balance check must still require an exact debit so other notes remain backed. Document which listed assets can change their transfer rules and what users can do if they do.

VALIDATION STEPS

L-1. Exact-delivery check in `_transferOut()` blocks withdrawals after a token fee switch

1. Check evm/src/DarkPool.sol:456-472: `_transferOut` requires both the pool's balance decrease and the recipient's balance increase to equal `amount`.

2. Check evm/src/DarkPool.sol:289-303 and 310-316: every token payout, relayer fee and fee sweep goes through `_transferOut`.

3. Check evm/src/DarkPool.sol:349-388: `listed` is never cleared and `updateAsset` only changes deposit flags and limits; there is no rescue function and no upgrade path.

4. Read the proof. The function below is taken verbatim from `test/DarkPoolFindingsPOC.t.sol`:

```
function test_L1a_RecipientSideTokenFeeLocksEveryNoteOfThatAssetWhileThePoolSt
    DarkPool pool = new DarkPool(address(new AcceptAllVerifier()), address(0),
    RecipientSurchargeToken token = new RecipientSurchargeToken();
    vm.prank(owner);
    pool.listAsset(address(token), 1e18, 1_000e18, 10_000e18);
    token.mint(alice, 1_000e18);
    vm.startPrank(alice);
    token.approve(address(pool), type(uint256).max);
```

```
    pool.shield(address(token), 100e18, 7, 8, ""); // exact transfer: accepted
    vm.stopPrank();
    assertEq(token.balanceOf(address(pool)), 100.5e18, "notes plus the 0.5% fe

    // The issuer switches the fee on. The pool still holds every unit it book
    token.enableFee();

    DarkPool.Proof memory p;
    p.root = pool.root();
    p.inputNullifiers = [uint256(1), uint256(2)];
    p.outputCommitments = [uint256(111), uint256(222)];
    DarkPool.ExtData memory e = DarkPool.ExtData({
        recipient: address(0xB0B), extAmount: -10e18, relayer: address(0), fee
        token: address(token), encryptedOutput1: "", encryptedOutput2: ""
    });
    vm.expectRevert(DarkPool.BadTransferAmount.selector);
    pool.transact(p, e);

    vm.expectRevert(DarkPool.BadTransferAmount.selector);
    pool.collectFees(address(token));

    // Nothing the owner can do reopens the exit: limits and the deposit flag
    // are the only knobs, and there is no delist, rescue or upgrade.
    vm.prank(owner);
    pool.updateAsset(address(token), false, 1e18, 1_000e18, 10_000e18);
    vm.expectRevert(DarkPool.BadTransferAmount.selector);
    pool.transact(p, e);

    assertEq(token.balanceOf(address(pool)), 100.5e18, "the pool is solvent in
    assertEq(noteBooks(pool, address(token)), 100e18, "the notes are intact an
}
```

5. Run the proof. Expected: 1 passed, 0 failed. The verifier is a stub that accepts every proof; the finding does not depend on the proof system.

```
forge test --match-path test/DarkPoolFindingsPOC.t.sol --match-test "test_L1a_
```

   Observed output on the commit above:

```
[PASS] test_L1a_RecipientSideTokenFeeLocksEveryNoteOfThatAssetWhileThePoolStay
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

6. The team's test evm/test/DarkPool.t.sol:695-704 exercises the same token shape and expects the revert, which is why this is reported as a policy consequence rather than an unnoticed bug.

FINDING DETAILS

Unchecked handover in Solana `set_admin()` can lock out the pool admin

Report ID: CUS-7

Status: Resolved

Severity: Low

TARGET

`https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md`

VULNERABILITY DETAILS

[L-2] Unchecked handover in Solana `set_admin()` can lock out the pool admin

Severity: Low

Risk Category: Functional correctness

Context

- **File**: solana/pool/src/lib.rs
- **Function**: `set_admin()`
- **Lines**: 617-627 (the new key is written directly)
- **Related**: evm/src/DarkPool.sol:50 (the EVM pool uses `Ownable2Step`)

Description

The EVM pool uses a two-step owner transfer. The Solana `set_admin` instruction replaces the admin key as soon as the current admin signs; the new key does not need to accept:

```
fn set_admin(program_id: &Pubkey, accounts: &[AccountInfo], r: &mut Reader) -> Pro
    let it = &mut accounts.iter();
```

```
    let admin = next_account_info(it)?;
    let pool = next_account_info(it)?;
    let new_admin: [u8; 32] = r.arr()?;
    r.done()?;
    check_pool(program_id, pool)?;
    check_admin(pool, admin)?;
    pool.try_borrow_mut_data()?[P_ADMIN..P_ADMIN + 32].copy_from_slice(&new_admin)
    Ok(())
}
```

A mistyped key, the zero key, or any key without a known signer leaves the current admin unable to act. A program-derived address is safe only if its program can sign for it. The lost role controls asset listings, deposit limits, and the deposit pause. Recovery would require intervention by the Solana upgrade authority while that authority still exists.

Impact

Users can still withdraw and make private transfers, but the team cannot list assets, change limits, or restore deposit policy through the existing admin key. If the upgrade authority has been removed, the lockout lasts for that deployment. The exposure is larger than the contract alone suggests: the setup manual presents the rotation as routine, the pool client exports no encoder for `set_admin` (nor for `update_asset`), so the rotation is assembled by hand, and `check-deployment.mjs` prints the admin as information with no expected value, so the manual's Part 12 has the operator hand-assemble the instruction and nothing would flag a botched one.

Proof of Concept

Scenario (code read):

1. The admin signs `set_admin` with `new_admin = [0u8; 32]` or any key not under their control.
2. `check_admin` accepted the signer, nothing validates `new_admin`, and the pool account now names a key nobody can sign for.
3. Every later `list_asset`, `update_asset`, `set_paused` and `set_admin` fails with `NotAdmin`.

No test accompanies this finding; the behavior is read directly from the function above.

Recommendation

Mirror the EVM contract's two-step handover: `set_admin` should reject the zero key and store a pending admin, leaving the current admin active until the new key accepts.

A separate `accept_admin` instruction signed by the pending key then moves it into `P_ADMIN`. The pool account has a single unused byte (offset 51) between `P_SOL_VAULT_BUMP` and `P_FEE_BPS`, so a pending-admin field needs a layout change, which is appropriate for a v3 candidate that already requires fresh deployments.

VALIDATION STEPS

L-2. Unchecked handover in Solana `set_admin()` can lock out the pool admin

1. Check solana/pool/src/lib.rs:617-627: `set_admin` copies `new_admin` into `P_ADMIN` after `check_admin`, with no validation of the new key and no acceptance step.
2. Check evm/src/DarkPool.sol:50: the EVM pool uses `Ownable2Step`.

No test accompanies this finding: the behavior is a single unconditional write that a runtime harness would only restate.

FINDING DETAILS

The guardian pause window is accepted unvalidated on both chains

Report ID: CUS-8

Status: Resolved

Severity: Low

TARGET

`https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md`

VULNERABILITY DETAILS

[L-3] The guardian pause window is accepted unvalidated on both chains

Severity: Low

Risk Category: Functional correctness

Context

- **File**: evm/src/DarkPool.sol
- **Lines**: 158-178 (the constructor validates the fee recipient, the verifier and sanctions code, and the fee cap, then stores `guardianExpiry_` raw at line 174), 384-388 and 392-394 (the semantics that make a bad value permanent)
- **File**: solana/pool/src/lib.rs
- **Lines**: 453 (`guardian_expiry` read as a raw i64), 459-460 (fee cap and zero-checks for fee recipient and genesis, none for the expiry), 483 (stored raw), 370-373 and 607-613 (the same permanent semantics)
- **File**: evm/script/Deploy.s.sol
- **Lines**: 106 (`GUARDIAN_DAYS` defaults to 30 with no bound), 65 (`block.timestamp + guardianDays * 1 days`, `0` is accepted and yields an already-expired window); the shell wrapper only checks that the value is a whole number

Description

The guardian window is the only constructor/initialize parameter on either chain with no validation at all. The EVM constructor rejects a zero fee recipient, a codeless verifier or sanctions oracle, and a fee above the cap; the Solana `initialize` rejects a fee above the cap and a zero fee recipient or genesis hash; the deploy script range-checks the fee, the deposit limits and the uint128 widths. The window itself is stored exactly as given: any past timestamp, zero, or `type(uint256).max` on the EVM, and any i64 including negatives on Solana, is accepted. The value is immutable for the life of each deployment, and both implementations derive every pause behavior from it: `setDepositsPaused(true)` reverts `PauseWindowOver` once `block.timestamp >= guardianExpiry`, `depositsPaused()` self-clears at the same instant, and Solana mirrors both. The wrapper's only guard is that `GUARDIAN_DAYS` is a whole number, so `0` and `36500` both pass.

Impact

Two one-way failure shapes. A window at or before deployment time (a `0` or mistyped `GUARDIAN_DAYS`, a past or negative timestamp) disables the deposit pause at birth: the documented owner power to "pause all deposits until guardianExpiry" is dead from block one. If an exploit appears or a malicious asset is listed, the operator cannot pause new deposits on that deployment. An oversized window (a misplaced digit, `36500`) inverts the design the contract comment promises ("a guardian cannot keep deposits paused by setting the flag shortly before its expiry"): the owner gains an effectively permanent deposit-censorship right for the pool's life, contradicting the time-limited guardian the contract describes (`DarkPool.sol:382-394`) and the architecture document's statement that deposit pauses expire. User funds, withdrawals and private transfers are unaffected in both shapes. The deployer supplies the value, but the missing validation is in both audited initialization paths and an ordinary mistake can permanently remove the emergency pause. A permanent loss of a safety control after an ordinary mistake by the trusted role is why Low is appropriate for this audit.

Proof of Concept

Scenario (code read; the reverting paths are asserted by the repository's own suites):

1. The deployer sets `GUARDIAN_DAYS=0` (the wrapper accepts it as a whole number). `Deploy.run` constructs the pool with `guardianExpiry == block.timestamp`.
2. `setDepositsPaused(true)` reverts `PauseWindowOver` (`block.timestamp >= guardianExpiry` holds from the first block) and `depositsPaused()` returns `false` unconditionally: the pause feature never existed on this deployment.
3. The Solana twin: `initialize` with `guardian_expiry = 1` (or any past/negative i64) passes every require at lines 459-461; `set_paused` then always fails `PauseWindowOver` and `deposits_paused_at` is always false.
4. The oversized shape: `GUARDIAN_DAYS=36500` stores an expiry a century out; the owner can pause deposits at will for the pool's life and the time limit the contract describes for the guardian does not exist.

Tests in the attached `test/DarkPoolFindingsPOC.t.sol`:

- `test_L3a_TheGuardianWindowIsStoredUnvalidatedOnTheEvm`

The test deploys pools with a window equal to the deployment time, with zero, and with a window a century out: the first two can never enable the pause, the third keeps deposits paused fifty years later. The Solana half is read from `initialize` and `set_paused`. The commands to run this test and the observed output are in validation-steps.md, section L-3.

Recommendation

Validate the window at the single point each chain records it, relative to the current time: on the EVM, `if (guardianExpiry_ <= block.timestamp || guardianExpiry_ > block.timestamp + MAX_GUARDIAN_WINDOW) revert BadExtData();` with a `MAX_GUARDIAN_WINDOW` constant chosen by the team (for example 90 days); on Solana the same bounds against `Clock::get()?.unix_timestamp` in `initialize`; and a matching range check for `GUARDIAN_DAYS` in `Deploy._readConfig` and the wrapper. These are fresh-deployment changes consistent with the v3 candidate's no-upgrade posture.

VALIDATION STEPS

L-3. The guardian pause window is accepted unvalidated on both chains

1. EVM: read `evm/src/DarkPool.sol:158-178`, the constructor validates feeRecipient!=0, verifier/sanctions code, feeBps≤100, then stores `guardianExpiry_` raw (line 174). Semantics at 384-388/392-394: a window ≤ deployment time makes `setDepositsPaused(true)` revert `PauseWindowOver` forever and `depositsPaused()` always false.
2. Solana: read `solana/pool/src/lib.rs:453` (raw i64 read), `459-461` (fee cap and zero-checks for recipient/genesis, none for the expiry), `482-483` (stored), `370-373` / `607-613` (same permanent semantics).
3. Deploy: `evm/script/Deploy.s.sol:106` (`GUARDIAN_DAYS` unbounded, default 30) and `:65` (`block.timestamp + guardianDays * 1 days` ; `0` accepted); `scripts/deploy-evm.sh` validates only "whole number".

Read the proof. The function below is taken verbatim from `test/DarkPoolFindingsPOC.t.sol`:

```
function test_L3a_TheGuardianWindowIsStoredUnvalidatedOnTheEvm() public {
    AcceptAllVerifier verifier = new AcceptAllVerifier();

    // Expired at birth: the pause can never be enabled.
    DarkPool dead = new DarkPool(address(verifier), address(0), block.timestamp, c
    vm.prank(owner);
    vm.expectRevert(DarkPool.PauseWindowOver.selector);
    dead.setDepositsPaused(true);
    assertFalse(dead.depositsPaused(), "no pause on a pool whose window expired at
```

```
    // Zero is accepted too, with the same effect.
    DarkPool zero = new DarkPool(address(verifier), address(0), 0, owner, feeRecip
    vm.prank(owner);
    vm.expectRevert(DarkPool.PauseWindowOver.selector);
    zero.setDepositsPaused(true);

    // A century out: the owner can pause deposits for the life of the pool.
    DarkPool forever = new DarkPool(address(verifier), address(0), block.timestamp
    vm.prank(owner);
    forever.setDepositsPaused(true);
    assertTrue(forever.depositsPaused());
    vm.warp(block.timestamp + 50 * 365 days);
    assertTrue(forever.depositsPaused(), "still paused fifty years later");
}
```

Run the proof. Expected: 1 passed, 0 failed.

```
forge test --match-path test/DarkPoolFindingsPOC.t.sol --match-test "test_L3a_TheG
```

Observed output on the commit above:

```
[PASS] test_L3a_TheGuardianWindowIsStoredUnvalidatedOnTheEvm()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

FINDING DETAILS

Informational

Report ID: CUS-9

Status: Informative

Severity: Low

TARGET

`https://github.com/hackenproof-public/darkswap-audit/blob/main/docs/zk-pool-audit/README.md`

VULNERABILITY DETAILS

[I] Fee routing, deployment checks, ETH payouts, and fee guidance need follow-up

Severity: Informational

Risk Category: Functional correctness, Griefing

The four observations below are independent. They are grouped for this audit's Informational section; their impacts do not add up to one exploit. None demonstrates loss of user principal.

Context

- **A. Solana fee destination:** `collect_fees`, lines 877-918, especially 909-915; the fee-recipient brief describes a future `Burner` vault.
- **B. Deployment verification:** `check-deployment.mjs:124-129`, `:189-191` and `:272`; `Deploy.s.sol:67-88` and `:111`; `devnet-setup.mjs:51-60` and `:136`. These scripts are supporting material outside the nine reviewed protocol files.
- **C. EVM ETH payout:** `DarkPool.transact`, lines 258-303 and `_transferOut`, lines 456-460; the relayer implementation is outside scope.
- **D. Fee guidance:** `docs/pool-earnings-models.md`, lines 177, 205 and 218, versus EVM `setProtocolFee`, lines 323-327 and Solana `set_fee`, lines 866-875. The v3 README marks

the old `REPLIT` handoff files historical, and the ceremony guide correctly says the rate cannot be changed after deployment.

Description

A. Caller-chosen SPL fee account. `collect_fees` is permissionless. For an SPL asset it checks that `destination` is a token account for the mint whose owner field equals the fixed fee recipient; it does not require that account to be the recipient's associated token account (ATA). A caller can create a different account for that mint with the fee recipient as token owner using `InitializeAccount3`, then direct a sweep there. The pool clears its accrued-fee entry. SOL sweeps do require the destination key itself to equal the recipient. The proposed `Burner` program is not implemented in this review, so whether it can spend from an arbitrary token account owned by its PDA is unknown.

B. Circular recipient check. The EVM deploy script reads `FEE_RECIPIENT` once, gives it to the immutable constructor and writes the same value to deployment JSON. The checker compares the live value with that JSON and with the other EVM deployment made from the same input. On Solana it prints a pass for whatever fee recipient the pool reports. The ceremony requires an independently reviewed manifest, but the checker never compares the live keys with it. A valid but unintended address can therefore pass the automated check. The error requires a deployment-input mistake; no live misconfiguration was shown.

C. Reverting ETH recipient. `transact` verifies the proof, marks nullifiers and inserts outputs before `_transferOut` makes an ETH call with all remaining gas. If the recipient reverts, the whole transaction rolls back. A withdrawer can choose a recipient whose `receive()` accepts a simulation at `tx.gasprice == 0` but rejects the relayer's mined transaction. The submitting relayer pays gas; the note and proof remain reusable. The project's relayer simulates without a gas price but throttles repeats until a prior attempt is final. This is an observation about the relayer's cost, not a loss to the pool.

D. Dated fee plan. The October 4 earnings plan describes `setProtocolFee` as a way to change a deployed rate and suggests testing a change. Both v3 setters always return `FeeImmutable` for an authorized caller. Current ceremony guidance is correct; the two older `REPLIT` files are explicitly historical. The remaining contradiction is in the dated planning document.

Impact

Observation A. A keypair recipient can recover fees from the alternate account. Fees become inaccessible only if the future `Burner` is immutable and can spend from its ATA alone. User notes and withdrawals are unaffected.

Observation B. A mistaken but valid recipient could receive all future protocol fees on that deployment. The scripts' apparent pass is the defect; this is outside the reviewed protocol-code scope and no such deployment was found.

Observation C. The relayer loses roughly 2.34 million gas in the local stub-verifier test (about 2.6 million with real verification); the pool rolls back correctly and user assets remain unchanged.

Observation D. A reader could plan around a fee setter that cannot work. There is no on-chain effect, and the current deployment guide states the correct rule.

Proof of Concept

Scenario (`test_I1a` for C, `test_I1b` for D; A and B by code and script reading):

1. A: create a non-ATA SPL token account with mint = listed asset and token owner = fee recipient, and pass it as `destination` to `collect_fees`. The checks at `lib.rs:909-915` pass, the fees move to it and `A_FEES` clears. No finding-specific Solana runtime test accompanies this observation.
2. B: put a wrong but valid address in `FEE_RECIPIENT` or `--fee-recipient`, deploy, then run `check-deployment.mjs`. Its comparisons use that same input or the pool's own value; none uses the manifest's intended address. No live deployment was tested.
3. C: a withdrawal to a recipient whose `receive()` requires `tx.gasprice == 0` succeeds in a zero-gas-price simulation, then reverts when mined at 1 gwei. About 2.34 million gas is charged with the stub verifier, nothing is consumed, and the same proof can be retried.
4. D: owner calls to `setProtocolFee` at 0, 25, 100 and the deployed rate all revert with `FeeImmutable`. The Solana setter is confirmed by source and the team's inline test.

Tests in the attached `test/DarkPoolFindingsPOC.t.sol`:

- `test_I1a_SimulationOnlyRecipientBurnsTheRelayersGasAndLeavesTheProofReusable`
- `test_I1b_EarningsPlanFeeChangeRevertsForEveryInput`

The commands to run these tests and the observed output are in validation-steps.md, section I-1.

Recommendation

- **A:** Before freezing the pool and `Burner`, either pin the ATA in `collect_fees` or make `Burner` able to spend from every token account its vault PDA owns.
- **B:** Record the independently approved recipient before deployment and compare each live chain value with that record; fail the checker on a mismatch.
- **C:** Have the relayer simulate with the gas price it will sign and restrict retries. A pool-level alternative is to defer a failed ETH payout to a pull balance while preserving the withdrawal and accounting for pending ETH.
- **D:** Update or mark the earnings plan superseded and replace its rate-change acceptance step with a check of the immutable deployed rate.

VALIDATION STEPS

I. Fee routing, deployment checks, ETH payouts, and fee guidance need follow-up

These are four independent Informational observations grouped in the report. Use the setup above for the two Foundry tests. The Solana fee-account and deployment-check observations are established by code reading and do not have finding-specific runtime tests.

A. Caller-chosen SPL fee account (code read). In solana/pool/src/lib.rs:877-918, `collect_fees` accepts a caller-supplied `destination` without a signer requirement. For an SPL asset, lines 909-915 check its mint and token-owner fields but not its ATA address. `InitializeAccount3` can initialize another account with the fee recipient as token owner without that owner's signature. The pool transfers the accrued amount there and clears `A_FEES`; SOL requires the recipient key directly. Whether this strands fees depends on the future `Burner` design, which is outside the review.

B. Circular fee-recipient check (script read). On EVM, `Deploy.s.sol:67-88` gives the same `FEE_RECIPIENT` input to the constructor and the deployment JSON; `check-deployment.mjs:124-129,189-191` compares those derived values. On Solana, line 272 prints a pass for the live recipient without an independent expected address. A wrong but valid input can therefore pass. The deployment scripts are outside the nine custom protocol files; no live misconfiguration was shown.

C and D (Foundry). Read the proofs. The 2 functions below are taken verbatim from `test/DarkPoolFindingsPOC.t.sol`:

```
function test_I1a_SimulationOnlyRecipientBurnsTheRelayersGasAndLeavesTheProofReusa
    DarkPool pool = new DarkPool(address(new AcceptAllVerifier()), address(0), blo
    listEth(pool);
    vm.deal(alice, 10 ether);
    vm.prank(alice);
    pool.shield{value: 1 ether + pool.feeOn(1 ether)}(ETH, 1 ether, 7, 8, "");

    SimulationOnlyRecipient trap = new SimulationOnlyRecipient();
    RelayerStandIn relayer = new RelayerStandIn();
    DarkPool.Proof memory p;
    p.root = pool.root();
    p.inputNullifiers = [uint256(61), uint256(62)];
    p.outputCommitments = [uint256(611), uint256(622)];
    DarkPool.ExtData memory e = DarkPool.ExtData({
        recipient: address(trap), extAmount: -0.5 ether, relayer: address(relayer)
        token: ETH, encryptedOutput1: "", encryptedOutput2: ""
    });

    // The relayer's simulation: gas price zero, the call succeeds.
    uint256 snapshot = vm.snapshotState();
    vm.txGasPrice(0);
    (bool simulated,) = relayer.submit(pool, p, e);
    assertTrue(simulated, "simulation says the withdrawal will go through");
    vm.revertToState(snapshot);

    // The broadcast: a real gas price, the recipient reverts, the whole
    // transaction unwinds after the nullifier writes and the pair insert.
    vm.txGasPrice(1 gwei);
    uint256 booksBefore = noteBooks(pool, ETH);
```

```
    (bool ok, uint256 gasUsed) = relayer.submit(pool, p, e);
    assertFalse(ok, "on chain the payout reverts");
    emit log_named_uint("gas burned by the failed attempt", gasUsed);
    assertGt(gasUsed, 2_000_000, "the relayer paid for the tree insert before the

    // Nothing was consumed: the same proof can be submitted again and again.
    assertFalse(pool.nullifierSpent(61), "first nullifier not spent");
    assertFalse(pool.nullifierSpent(62), "second nullifier not spent");
    assertEq(pool.nextIndex(), 2, "no outputs stored");
    assertEq(noteBooks(pool, ETH), booksBefore, "note books untouched");
    assertEq(address(relayer).balance, 0, "no relayer fee was paid");
    assertEq(address(trap).balance, 0, "the attacker spent nothing");
    (bool again, uint256 gasAgain) = relayer.submit(pool, p, e);
    assertFalse(again, "the second attempt fails the same way");
    assertGt(gasAgain, 2_000_000, "and costs the relayer the same again");
}
```

```
function test_I1b_EarningsPlanFeeChangeRevertsForEveryInput() public {
    DarkPool pool = new DarkPool(address(new AcceptAllVerifier()), address(0), blo
    uint16[4] memory rates = [uint16(0), 25, 100, FEE_BPS]; // the earnings plan's
    for (uint256 i = 0; i < rates.length; i++) {
        vm.prank(owner);
        vm.expectRevert(DarkPool.FeeImmutable.selector);
        pool.setProtocolFee(rates[i]);
    }
    assertEq(pool.protocolFeeBps(), FEE_BPS, "the rate chosen at deployment is the
}
```

Run them. Expected: 2 passed, 0 failed.

```
forge test --match-path test/DarkPoolFindingsPOC.t.sol --match-test "test_I1a_Simu
```

Observed output on the commit above:

```
[PASS] test_I1a_SimulationOnlyRecipientBurnsTheRelayersGasAndLeavesTheProofReusabl
  gas burned by the failed attempt: 2340078
[PASS] test_I1b_EarningsPlanFeeChangeRevertsForEveryInput()
Suite result: ok. 2 passed; 0 failed; 0 skipped
```

In C, with a stub verifier, a zero-gas-price simulation succeeds, a mined attempt at 1 gwei reverts after the tree insert and charges the submitting relayer 2,340,078 gas. Nullifiers, notes, books and fee balances roll back; the same proof can be retried. The team's relayer code used for this scenario is outside scope.

In D, owner calls with 0, 25, 100 and the deployed rate all revert with `FeeImmutable`; the rate stays fixed. The Solana `set_fee` path also returns `FeeImmutable` after its admin check, as the

team's inline test asserts. `docs/pool-earnings-models.md:177-218` still describes a change, while the v3 ceremony guide correctly says the rate is immutable.

ABOUT PROJECT

DarkSwap is in beta. A “private” or “confidential” route is not a guarantee of anonymity or unlinkability. Solana deposits are public on-chain. Review the live quote, destination, asset, amount, address, memo, and deadline before sending anything.

DISCLAIMER

HackenProof Disclaimer

The application provided for assessment has been analyzed in accordance with generally accepted industry practices applicable at the time this report was prepared. The assessment focused on identifying cybersecurity vulnerabilities and security issues in the application’s

source code, deployment, configuration, and functionality, based solely on the materials, access, and scope made available for the engagement.

This report does not provide any representation or warranty that all vulnerabilities, weaknesses, or misconfigurations have been identified, nor does it guarantee the absolute security, reliability, or bug-free status of the application. The conclusions herein apply only to the specific version and scope reviewed and may no longer remain valid after any modification, update, or environmental change. This report should not be regarded as a definitive or exhaustive statement of the application’s security posture.

Technical Disclaimer

Applications, whether decentralized applications (DApps) or other software systems, are deployed and operated within technical environments that may include blockchains, cloud services, operating systems, programming languages, frameworks, libraries, APIs, third-party services, and infrastructure components. Any of these elements may contain inherent weaknesses, undisclosed vulnerabilities, or configuration issues that can result in security incidents, operational failures, or unauthorized access.

The assessment reflects the condition of the application only at the time of testing and within the agreed scope, and the absence of findings in this report must not be interpreted as evidence that no vulnerabilities exist. Continuous security practices, including secure development, independent reviews, monitoring, patch management, and periodic reassessments, remain necessary to maintain an appropriate level of security.



HackenProof

Expert crowdsourced security provider

PRIVATE SECURITY AUDIT

Contact us:

Linkedin | X | Website | Mail

2026